

The logo of Gazi University is a circular seal. It features the university's name in Turkish, "GAZİ ÜNİVERSİTESİ", around the top inner edge and the year "1926" at the bottom. In the center is a stylized signature of "Gazi".

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

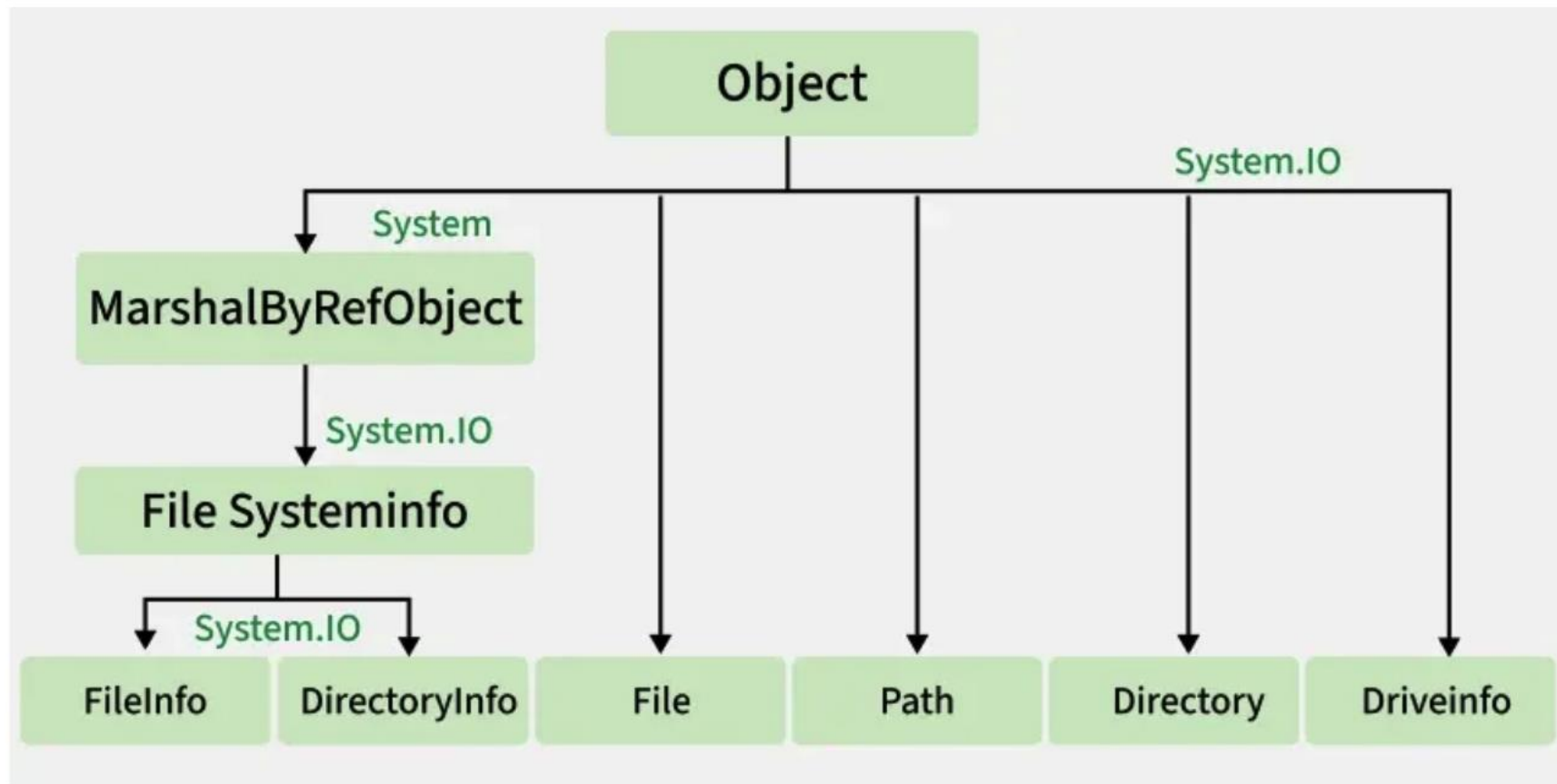
Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

WEEK 12: FILE AND EXCEPTION HANDLING

FILE HANDLING

File handling in C# allows developers to create, read, write and manage files stored on disk. The .NET Framework provides the System.IO namespace, which contains classes and methods for performing these operations in a simple and efficient way.



File Handling classes in C#

FILE HANDLING CLASSES

C# uses the System.IO namespace for file and directory operations. Commonly used classes include:

- **File:** Provides static methods for creating, copying, deleting, moving and opening files.
- **FileInfo:** Provides instance methods for file operations with additional object-oriented features.
- **Directory:** Provides static methods for creating, moving and enumerating directories.
- **DirectoryInfo:** Provides instance methods for directory operations.
- **StreamReader / StreamWriter:** For reading and writing text files.
- **BinaryReader / BinaryWriter:** For reading and writing binary files.
- **FileStream:** For reading and writing to files at a lower level.

CREATING A FILE

You can create a file using the File class or the FileStream class.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string path = "sample.txt";
        File.WriteAllText(path, "Hello, File Handling in C#!");
        Console.WriteLine("File created successfully.");
    }
}
```

Another method, **File.WriteAllLines**, is used to write an array of strings as individual lines. In contrast, **File.WriteAllText** writes a single string exactly as provided, without adding any extra formatting or line breaks.

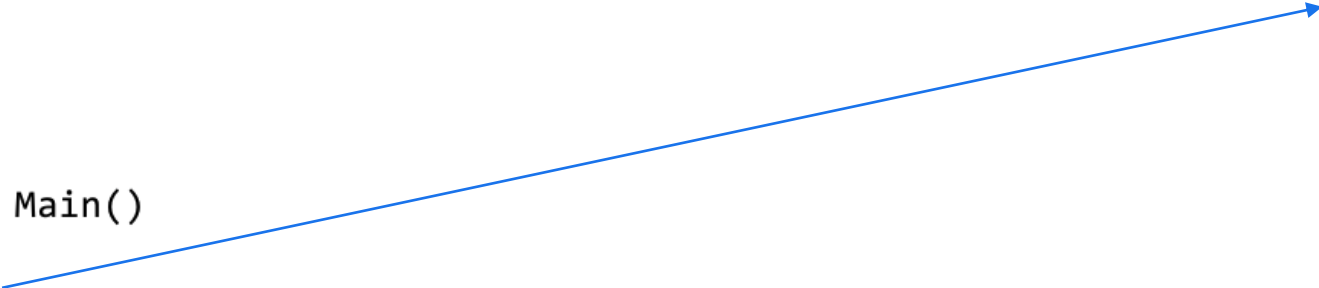
- File.WriteAllText creates the file if it doesn't exist and writes the given string to it.
- If the file already exists, its contents are overwritten.

WRITING LINES TO A FILE

You can write text data using StreamWriter.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        using (StreamWriter writer = new StreamWriter("data.txt"))
        {
            writer.WriteLine("Line 1");
            writer.WriteLine("Line 2");
        }
        Console.WriteLine("Data written to file.");
    }
}
```



The **using** keyword creates an object within a specific block of code, assigns it to a variable, and ensures it is automatically closed once the block finishes.

- StreamWriter is wrapped in a using block to ensure the file is closed automatically.
- WriteLine writes each string to a new line in the file.

READING LINES FROM A FILE

You can read text files using StreamReader.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        using (StreamReader reader = new StreamReader("data.txt"))
        {
            string content = reader.ReadToEnd();
            Console.WriteLine("File Content:");
            Console.WriteLine(content);
        }
    }
}
```

Now try to write this example with ReadLine!

- ReadToEnd reads the entire file content into a single string.
- Alternatively, you can use ReadLine to read line by line.

APPENDING NEW LINE TO FILE

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        // Appending to the file
        File.AppendAllText("data.txt", "\nAppended Line");
        Console.WriteLine("Line appended.");

        // Reading the content to verify the append
        using (StreamReader reader = new StreamReader("data.txt"))
        {
            string content = reader.ReadToEnd();
            Console.WriteLine("File Content:");
            Console.WriteLine(content);
        }
    }
}
```

Appending adds data without deleting existing content.

What is the output of this code?

MANAGING FILE PATHS

It is crucial to manage paths to ensure that your application can locate and access files accurately. The .NET framework provides useful methods in the **System.IO.Path** class for manipulating file and directory paths.

Combine Paths:

The **Path.Combine** method combines multiple strings into a single path.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string folderPath = "C:\\Users\\User";
        string fileName = "example.txt";
        string fullPath = Path.Combine(folderPath, fileName);
        Console.WriteLine(fullPath); // Output: C:\Users\User\example.txt
    }
}
```


GET FILE NAME AND EXTENSION

Getting the File Name:

The `Path.GetFileName` method returns the file name along with its extension.

```
string filePath = "C:\\Users\\User\\example.txt";  
string fileName = Path.GetFileName(filePath);  
Console.WriteLine(fileName); // Output: example.txt
```

Getting the File Extension:

The `Path.GetExtension` method returns the extension of a file, including the dot.

```
string filePath = "example.txt";  
string extension = Path.GetExtension(filePath);  
Console.WriteLine(extension); // Output: .txt
```

Checking the Existence of Files and Directories

- When working with files or directories in C#, it's always a good practice to check if they exist beforehand.
- .NET framework offers built-in methods to perform these checks efficiently.

Check Whether a File Exists:

The `System.IO.File.Exists` method checks if a file exists at a specified path. Consider that we have using `System.IO` added above the class for the following code:

```
string filePath = "C:\\Users\\Logs\\example.txt";

if (File.Exists(filePath))
{
    Console.WriteLine("File exists.");
}
else
{
    Console.WriteLine("File does not exist.");
}
```

Checking the Existence of Files and Directories

Check Whether a Directory Exists:

The `System.IO.Directory.Exists` method checks if a directory exists at a specified path. Again, consider that we added `System.IO` at the beginning:

```
string directoryPath = "C:\\Users\\Logs";

if (Directory.Exists(directoryPath))
{
    Console.WriteLine("Directory exists.");
}
else
{
    Console.WriteLine("Directory does not exist.");
}
```

EXCEPTION HANDLING

It is the process of responding to runtime anomalies, known as exceptions, in a controlled way. Proper handling ensures that a program continues to run or exits gracefully instead of crashing unexpectedly. C# provides structured exception handling using the keywords try, catch, finally, and throw.

Syntax:

```
try{  
    // Code that may throw an exception  
}  
  
catch (ExceptionType ex){  
    // Code to handle the exception  
}  
  
finally{  
    // Code that will always execute  
}
```

Keywords for Exception Handling

1. try

The try block contains the code that might throw an exception. Only the code inside the try block is monitored for exceptions.

```
using System;

class Program
{
    static void Main()
    {
        try
        {
            int x = 0;
            int result = 10 / x;
            Console.WriteLine(result);
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Division by zero detected.");
        }
    }
}
```

- The division uses a variable (x), so the error occurs at runtime, allowing the try-catch block to handle the *DivideByZeroException*. By using try and catch, program will be controlled against crashes.

Keywords for Exception Handling

2. catch

The catch block handles exceptions thrown in the try block. You can have multiple catch blocks to handle different exception types.

```
try{
    int[] arr = { 1, 2, 3 };
    Console.WriteLine(arr[5]); // IndexOutOfRangeException
}
catch (IndexOutOfRangeException ex){
    Console.WriteLine("Array index is out of range.");
}
catch (Exception ex){
    Console.WriteLine("Some other exception occurred.");
}
```

- The first matching catch block handles the exception.
- The second catch (Exception) acts as a fallback for any other exceptions.

Keywords for Exception Handling

3. finally

The finally block is mainly used for cleanup tasks such as closing files, releasing unmanaged resources, or freeing external connections.

```
try
{
    int x = 10, y = 0;
    int result = x / y;
}
catch (DivideByZeroException)
{
    Console.WriteLine("Cannot divide by zero.");
}
finally
{
    Console.WriteLine("Execution completed.");
}
```

- Even if no exception occurs, the code in finally runs.
- Ensures essential cleanup operations are performed.

Keywords for Exception Handling

4. throw

The throw keyword is used to manually raise an exception. It can be used to signal that an error or invalid condition has occurred.

```
int age = 15;

if (age < 18)
{
    throw new ArgumentException("Age must be 18 or older.");
}
```



- throw creates an exception object and passes it up the call stack.
- The exception can be caught by a try-catch block in the calling code.

Example-1: Handling Division by Zero

```
using System;

class Program
{
    static void Main()
    {
        int a = 10, b = 0;
        try
        {
            int result = a / b; // May throw DivideByZeroException
            Console.WriteLine(result);
        }
        catch (DivideByZeroException ex)
        {
            Console.WriteLine("Error: Division by zero is not allowed.");
        }
        finally
        {
            Console.WriteLine("Execution of try-catch block finished.");
        }
    }
}
```



Output:

*Error: Division by zero is not allowed.
Execution of try-catch block finished.*

Example 2: Multiple Catch Blocks

```
try
{
    int[] arr = { 1, 2, 3 };
    Console.WriteLine(arr[5]); // Throws IndexOutOfRangeException
}
catch (DivideByZeroException)
{
    Console.WriteLine("Division by zero error.");
}
catch (IndexOutOfRangeException)
{
    Console.WriteLine("Array index is out of range.");
}
catch (Exception)
{
    Console.WriteLine("Some other exception occurred.");
}
```

OUTPUT: *Array index is out of range.*

Example: Exercise

Now your turn to create the following program:

1. **Initialize Environment:** Define a constant string for your file path.
2. **Input Collection:** Create a method that initializes an array of a specific size, loops through that size to collect user strings via `Console.ReadLine()`, and returns the filled array.
3. **Start Safety Block:** Open a try block to protect your code from crashes during disk access.
4. **Write Data:** Use a built-in method to write the entire array to the file path at once.
5. **Read Data (Resource Management):** Open a using block with a `StreamReader` to ensure the file handle is released even if an error occurs.
6. **Iteration:** Inside a loop, check if the "end of stream" has been reached; if not, read one line at a time and print it to the console with a prefix.
7. **Error Handling:** Add a catch block specifically for `IOException` to display a clean message to the user if the file is locked or the disk is full.

```
using System;
using System.IO;
```

```
class Program
{
    static void Main()
    {
        string path = "data.txt";
        string[] inputs = ..... (3)

        ....
        {
            File. .... (path, inputs);
            Console.WriteLine("File saved. Reading back contents:");

            using (StreamReader reader = new StreamReader(path))
            {
                while (!reader. .... )
                {
                    Console.WriteLine($"Row: {reader.ReadLine()}");
                }
            }
        }
        .... (IOException ex)
        {
            Console.WriteLine($"Error accessing file: {ex.Message}");
        }
    }

    static ..... (int count)
    {
        string[] strArray = new string[count];
        for ( ..... )
        {
            Console.Write($"Enter item {i + 1}: ");
            strArray[i] = Console. .... ();
        }
        return strArray;
    }
}
```

Use this optional scaffolding to begin with coding.

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM